EXAMEN FINAL DE EDI II
Apellido y Nombre: Noelia Paulina Peralta
DNI: 38.845.149
2do año

Contenidos a Evaluar: Programación Estructurada. Control de Flujo. Manejo de Archivos. Uso de Librerías.

Criterios de Evaluación:

- Se evaluará ortografía.
- Se evaluará comprensión de consignas.
- Precisión en la utilización de lenguaje técnico.
- Utilización de la Programación.

Criterios de Acreditación:

- 0 a 59 - REPROBADO
- 60 a 63 NOTA 4 - APROBADO
- 64 a 69 NOTA 5 - APROBADO
- 70 a 76 NOTA 6 - APROBADO - 77 a 83 NOTA 7 - APROBADO
- 84 a 89 NOTA 8 - APROBADO - 90 a 96 NOTA 9 - APROBADO
- 97 a 100 NOTA 10 – APROBADO **Consignas:**

Ejercicios con lambda: *(25 puntos).*

Realizar una función lambda que eleve al cuadrado un número pasado como parámetro. Utilice la función map para mapear una lambda expresión que retorne la potencia de 2 de cada número en la lista números. El resultado (la potencia de cada número) se debe ir almacenando en una nueva lista.

Notas:

1. Realizar la lambda expression "in line", es decir, no declarar la lambda fuera del map sino directamente dentro. Copiar la lambda creada en el paso anterior dentro del map.
2. No debe usar "potencia_2" dentro del map, debe colocar directamente la lambda.

Ejercicios comprensión de listas: *(25 puntos).*

Generar una lista a partir de comprensión de listas, esta lista generada deberá contener la tabla del 5, desde el múltiplo 0 al múltiplo 10. El resultado esperado es: [0 5 10 15 20 25 30 35 40 45 50].

Utilizar comprensión de listas para generar esta lista. Lo esperable es que realicen una lista de 11 elementos, del 0 al 10 pero que cada elemento lo multipliquen x 5. Resultado esperado: tabla_5 = [.....].

Adjuntar como respuesta a este examen todos los archivos necesarios para su prueba.

Cuestionario (cada pregunta deberá ser contestada con sus propias palabras):

1. Según su criterio las 10(diez) librerías Python más importantes a tener en cuenta para la realización de sus proyectos, fundamente cada una de ellas. *(10 puntos).*
2. Existe otra forma más avanzada para imprimir en consola que el método “format”, los f-string, explicarlos y brindar algunos ejemplos. *(10 puntos).*

3. Tupla versus Lista en Python: ¿Cuál es la diferencia? ¿Podría explicarlo en breves palabras? *(10 puntos).*

4. En Python no existe el ciclo do-while. De hecho, no es necesario. ¿Podrías explicar por qué? *(10 puntos).*

¿Cómo podría organizar los departamentos de los pisos existentes en edificios con Python? Dar un ejemplo. *(10 puntos).*

Desarrollo:

1)

1. **NumPy:** Fundamental para trabajar con grandes volúmenes de datos numéricos, facilita operaciones matemáticas rápidas sobre arrays, base para muchas otras librerías.
2. **Pandas:** Permite organizar, limpiar y analizar datos tabulares de forma eficiente con estructuras como DataFrames, muy usado en ciencia de datos.
3. **Matplotlib:** Es la base para hacer gráficos en Python, ayuda a visualizar datos para entender resultados y patrones.
4. **Seaborn:** Construida sobre Matplotlib, facilita crear gráficos estadísticos más atractivos y con menos código.
5. **Scikit-learn:** Brinda herramientas para machine learning clásico: clasificación, regresión, clustering, muy usada para prototipos rápidos.
6. **TensorFlow:** Framework potente para construir modelos avanzados de deep learning, usado en inteligencia artificial.
7. **Requests:** Facilita la comunicación con páginas web y APIs, imprescindible para consumir servicios externos.
8. **BeautifulSoup:** Herramienta para extraer información de páginas web, útil para obtener datos donde no hay APIs disponibles.
9. **Flask:** Framework ligero para crear aplicaciones web o APIs, ideal para proyectos pequeños o prototipos.
10. **Pytest:** Simplifica la creación y ejecución de tests automáticos para asegurar que el código funciona correctamente y evitar errores.

2) Las **f-strings** (formatted string literals) son una forma moderna y más sencilla de formatear cadenas en Python (desde la versión 3.6). Permiten insertar expresiones dentro de las cadenas usando llaves {}, son más legibles y rápidas que .format(), y se escriben anteponiendo una f antes de las comillas.

Ejemplo con .format():

```
nombre = "Noelia"
edad = 30
print("Mi nombre es {} y tengo {} años".format(nombre, edad))
```

Ejemplo con f-string:

```
nombre = "Noelia"
edad = 30
print(f"Mi nombre es {nombre} y tengo {edad} años")
```

Entre las ventajas de f-strings podemos destacar que son más cortas y claras y se pueden poner expresiones dentro como:

```
print(f"El doble de 5 es {5*2}")
```

Además, soportan formatos como:

```
pi = 3.14159
print(f"Pi con 2 decimales: {pi:.2f}")
```

3) Las **listas** y las **tuplas** en Python almacenan colecciones de datos, pero las **listas** son mutables, es decir, que se pueden cambiar sus elementos después de crearlas (agregar, eliminar o modificar). Se escriben con corchetes [ ].
Las **tuplas**, en cambio, son inmutables: una vez creadas, no se pueden modificar sus elementos ni su tamaño. Se definen con paréntesis ( ). Por eso, las **tuplas** son útiles para datos que deben permanecer constantes y suelen ser un poco más rápidas que las **listas**.

4) En Python no existe el ciclo **do-while** porque su funcionalidad se puede replicar fácilmente usando un ciclo **while** normal.
El ciclo do-while ejecuta el bloque de código al menos una vez y luego verifica la condición para repetirlo. En Python, se puede hacer con un **while True** y una condición interna para salir con **break**. Así se puede asegurar de que el código se ejecute al menos una vez y luego decida si sigue o no.

Por ejemplo:

```
while True:
    # Código que se ejecuta al menos una vez
    if condición_de_salida:
        break
```

Esto elimina la necesidad de un do-while específico, manteniendo el código claro y simple.

5) Se puede usar una estructura de datos como un **diccionario** para organizar los departamentos por piso en un edificio. En el diccionario, cada clave sería el número del piso y el valor una lista con los departamentos de ese piso.
Por ejemplo:

```
edificio = {
    1: ["Depto 101", "Depto 102", "Depto 103"],
    2: ["Depto 201", "Depto 202"],
    3: ["Depto 301", "Depto 302", "Depto 303", "Depto 304"]
}

# Mostrar departamentos del piso 2
print("Departamentos del piso 2:", edificio[2])
```

Así se puede acceder fácilmente a los departamentos de cada piso y también agregar o modificar otros.